

GO PROGRAMMING MASTER COURSE

Week 4, Day 1

Modules, packages, and project shape —
today the Student Management System clock
starts.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST
WEEK

Prove the katas, then start the product

- Week 3 homework: green go test on Contains, Unique, Freq, and friends.
- Cold-call: why `s = append(s, x)`? What is the backing-array gotcha?
- `gofmt` still required. Secrets still never go in git.
- Today we stop writing throwaway folders and start the SMS repo layout.

TODAY

If Day 1 of week 4 works, you can

- Explain package vs module in one sentence each.
- Run `go mod init` and read a `go.mod`.
- Use `internal/` as a boundary, not a random folder name.
- Scaffold the SMS backend tree used in this course.
- Ship a README that states the product in three sentences.
- Open a PR with a clean skeleton — no `.env`, no binaries.

SOFTWARE
ENGINEERING

The product clock starts today

Weeks 1–3 were language fuel. From now on every week lands something in **SMS** — the campus admin API you will demo in week 16.

- Today: empty module + folders + README. Not an HTTP server yet.
- Spiral teaching: just enough layout to grow handlers next week.
- One repo per student. Instructor is collaborator. Homework is a **PR**.

PACKAGES

A package is a folder of Go files with the same name

```
// greeter/greeter.go
package greeter

func Hello(name string) string {
    return "Hello " + name
}
```

- All .go files in one directory share one package name.
- package main + func main = a program you can go run.
- Other packages are libraries. You import them; you do not go run them alone.

IMPORTS

Import path = module path + folder

```
import (  
    "fmt"  
    "student-management-system"  
)
```

- Stdlib has short paths: `fmt`, `strings`, `net/http`.
- Your code uses the module name from `go.mod`, then the directory.
- Wrong import path → “package ... is not in std” if you forgot `go.mod`.

EXPORT
RULES

Capital letter = visible outside the package

Exported

Hello, Student, NewHandler —
other packages can use these
names.

Unexported

hello, parseEnv, db — private
to this package. Prefer private by
default.

Same package

Files in one folder see each
other's unexported names. Tests
in the same package can too.

Not Java public

There is no `public` keyword. The
first letter *is* the rule.

INTERNAL/

Go's built-in "do not import me from outside"

```
backend/  
cmd/server/          #  
internal/handlers/   #  
// another module else
```

- Anything under a directory named `internal` is only importable by the tree rooted at the parent of `internal`.
- That is a compiler rule, not a style guide. Use it as the SMS boundary.
- Put product code in `internal/`. Put the process entry in `cmd/`.

MODULES

go mod init names the project

```
mkdir -p ~/epic-go/sms/backend  
go mod init student-management-  
cat go.mod
```

- Creates go.mod with a module path. That path prefixes your imports.
- Week 2–3 used modules as a ritual. Today we teach what it *means*.
- One module for the SMS backend is enough for this course.

GO.MOD

The file that defines your module

```
module student-manage
```

```
go 1.25
```

- module line = import prefix for your packages.
- go line = language version you target (pin **1.25.x** in class).
- Later: require lines appear when you go get Gin, pgx, etc. Not today.

TOOLS ·
LIGHT

go get and go mod tidy

- `go get example.com/mod@v1.2.3` — add or upgrade a dependency.
- `go mod tidy` — add missing requires, remove unused ones. Run it before you push.
- Today you may have **zero** third-party deps. That is fine. Tidy still keeps `go.mod` honest.

```
go mod tidy
go build ./
```

LAYOUT

The SMS backend tree (memorise the jobs)

```
backend/  
  cmd/server/main.go      # process entry  
  internal/config/         # env / settings  
  internal/handlers/       # HTTP adapters  
  internal/repository/     # data access  
  internal/models/         # structs / domain  
  internal/routes/         # wire URLs to handlers  
  internal/middleware/     # JWT later  
  internal/database/       # Postgres later  
  internal/utils/          # JWT helper later  
go.mod  
README.md
```

CMD/SERVER

One process entry — keep it thin

```
// cmd/server/main.  
package main  
  
import "fmt"  
  
func main() {  
    fmt.Println("sm  
}
```

- cmd/<name> is the usual Go pattern for executables in a repo.

- Today the main only proves the module builds. Next week it will call routes.

- Run: go run ./cmd/server from backend/.

INTERNAL
PACKAGES

One job per folder

handlers

Talk HTTP. Translate request → call something else → JSON.

repository

Talk data. Maps today, SQL later. No Gin imports here.

models

Shapes: Student, User. No business workflow.

config

Read env. Fail fast if required secrets missing (later).

LIVE
DEMO

SCAFFOLD

Type the skeleton with me

```
cd ~/epic-go/sms/backend
go mod init student-management-system
mkdir -p cmd/server internal/{config,handlers,repository,m
printf '%s\n' 'package main' 'import "fmt"' 'func main() {
    > cmd/server/main.go
gofmt -w cmd/server/main.go
go run ./cmd/server
```

CLEAN
CODE

Small files, clear names, no junk drawer

- A package name is a noun for its job: handlers, not stuff.
- Prefer many small files over one 800-line `main.go`.
- We still have `internal/utls` for JWT later — that is a **tradeoff**, not a virtue. Do not dump random helpers there this week.
- `gofmt` on save. Unformatted PRs are returned.

README

The contract of the repo

- Three sentences: what SMS is, who uses it, what “done” looks like this term.
- How to run (even if today it only prints sms starting).
- Module path, Go version, link to your GitHub.
- A README that says “final year project” and nothing else fails the homework.

```
# Student Management System
Staff register/login, then manage
This repo is the Epic Learn course
```

GIT / PR Homework is a pull request, not a zip

```
git checkout -b week-04-skeleton
git add .
git commit -m "Add SMS backend s
git push -u origin week-04-skeleton
# open PR on GitHub → invite instructor
```

- Branch names: short, purposeful.

- PR description: what and why (three lines is enough today).

- Instructor merges only if gofmt is clean and .env is absent.

SECRETS

**.env stays local.
.env.example is
committed.**

```
# .gitignore
.env
*.exe
/bin/
```

```
# .env.example (s
PORT=8080
DATABASE_URL=
JWT_SECRET=
```

- Empty values in the example. Real secrets never in git history.
- We do not need a real .env yet — create the ignore rule now so week 5 is safe.

Done when all of this is true

- `backend/go.mod` exists with module `student-management-system`
- Folder tree matches the slide (empty `internal/*` folders are fine)
- `go run ./cmd/server` prints `sms starting`
- README has three product sentences + how to run
- `.env` is gitignored; no binaries committed
- Branch pushed; PR URL shown to instructor

HOMEWORK

PR the skeleton before next class

- Same gate as the lab — finished and pushed.
- PR title example: Week 4 – SMS backend skeleton.
- Optional: one paragraph in the PR on why internal/ exists.
- Do **not** add Gin or HTTP yet. That is week 5.

**NEXT
SESSION**

Week 5 — HTTP, JSON, Gin, first SMS API

- Stdlib net/http hello, then Gin.
- GET /health → JSON `{"status": "ok"}`.
- Bring a merged or reviewable skeleton PR.
- Postman arrives. The API becomes real.

WORDS
TO KEEP

**Say these
correctly next
week**

Package

Folder of Go files with one package clause.

Module

Versioned unit defined by go.mod; import prefix.

internal/

Compiler-enforced privacy boundary for your app code.

cmd/

Place for main packages / executables.

Questions